ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

CyberSecurity

4 - "User Authentication"

[Chapter 3]

Gabriele D'Angelo <g.dangelo@unibo.it>

https://www.unibo.it/sitoweb/g.dangelo/en

Outline

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Means** of Authentication
- **Password**-based Authentication
- **Token**-based Authentication
- **Biometric** Authentication
- **Remote** User Authentication
- **Security Issues** for User Authentication

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Authentication

RFC 2828: User Authentication

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

The system does not take your word for it (your claim) but requires proof. The verification process serves to link the entity making the claim to the identity it has declared, usually through something the entity knows (a password), has (a token), or is (biometrics).

An entity can be: a human user attempting to log in, a software process or microservice that needs to communicate with another server, or a hardware device (such as an IoT sensor) that connects to a network.

declared by or on behalf of

The **process** of **verifying an identity** claimed by or for a system entity

Authentication never starts from scratch; it always begins with a claim. When you enter a username (e.g., mario.rossi), you are telling the system: “Hey, I affirm that I am Mario Rossi.” Identity can be declared by the entity itself (“by”) or by another system on its behalf (“for”), as is the case, for example, in Identity Provider protocols (OAuth).

Authentication Process: the process of verifying an entity's identity

Authentication Process

- Fundamental building block and **primary line of defense** of the entire security system
- Basis for **access control** and **user accountability**

- Access Control: The system cannot determine what you are permitted to do (Authorization) unless it first knows with absolute certainty who you are (Authentication).
- User Accountability: In the event of anomalies or attacks, you must be able to trace a specific action back to a specific user. Without strong authentication, actions become anonymous and can be repudiated.

- **Typically** composed of **two separate steps**:
  1. **identification step**: "*presenting an **identifier** to the security system*"

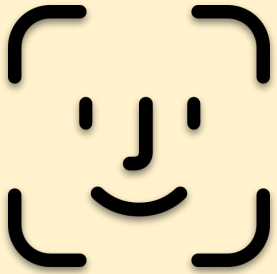
Examples: Enter your username or your email address. At this stage, the system only knows that someone is claiming the identity associated with that identifier.

  2. **verification step**: "***presenting*** *or* ***generating*** *authentication information that ~~corroborates~~ confirms the* ***binding*** *between the* ***entity*** *and the* ***identifier*** *link*"

Examples: Entering a password (something you know). Generating an OTP (One-Time Password) using a cryptographic algorithm on your smartphone (something you have) based on a shared secret key. If the verification is successful, the binding is confirmed and the system considers you authenticated.

Authentication Process

On your personal phone, the identification step is automatic (the phone already knows it belongs to a single owner). As a result, your face is your password (the verification factor). It works exactly like a PIN: the phone scans your face, converts it into a mathematical value, and compares it to the one stored in its memory. If they match, the "password" is correct and the phone unlocks.



Face ID: is your face your identifier or your password?

- Fundamental building block and **pri**
- Basis for **access control** and **user**
- **Typically** composed of **two**
  - **identification ste**: *"presenting an **identifier** to the security system"*
  - **verification step**: *"**presenting** or **generating** authentication information that corroborates the **binding** between the **entity** and the **identifier**"*

This slide introduces the four pillars of authentication, that are the categories of information a system may require to verify a user's identity. In technical terms, these are called authentication factors.

User Authentication

the **four** methods for authenticating user identity are based on:

1 something the individual **knows**

- *password, PIN, answers to pre-arranged* (pre-determined) *questions*

2 something the individual **possesses** (token)

- *smartcard, electronic keycard, physical key*

3 something the individual **is** (static biometrics)

- *fingerprint, retina, face*

4 something the individual **does** (dynamic biometrics)

- *voice pattern, handwriting, typing rhythm*

User Authentic

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

the **four mean** entity

The password recovery questions (i.e., security questions): they must be as secure as the password

something the individual **knows**

- *password, PIN, answers to pre-determined pre-arranged questions*

some… individ… **possesses**

- *…card, electronic keycard, physical key*

…biometrics)

- *fingerprint, retina, face*

…g the …al **does** (dynamic biometrics)

- *voice pattern, handwriting, typing rhythm*

User Authentication

the **four means** of authenticating user identity are based on:

something the individual **knows**

- *password, PIN, answers to pre-arranged questions*

something the individual **possesses** (token)

- *smartcard, electronic keycard, physical key*

something the individual **is** (static biometrics)

- *fingerprint, retina, face*

something the individual **does** (dynamic biometrics)

- *voice pattern, handwriting, typing rhythm*

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Passwords

Password Authentication

1. Privileges (System-Wide Capabilities): A user's general capabilities and roles within the system, independent of any specific object.
- Core Question: "What types of actions is this user authorized to perform?"
- What they are: Roles, profiles, or macro-permissions (e.g., Reader, Writer, Administrator).
- Example: User ID mario.rossi has "Write" privileges and belongs to the Human Resources group.
2. Access Control (Resource-Specific Verification): The practical enforcement process performed every time a user attempts an action on a specific resource (file, database table, web page).
- Core Question: "Does this specific User ID have the right to perform this specific action on this specific object?"
- What it is: The intersection of the user's privileges and the resource's security policy (e.g., Access Control Lists - ACLs).
- Example: User mario.rossi tries to open stipendi_manager.pdf. The system checks the file's rules: " Access restricted to Executives only". Since Mario is in HR, the system enforces ACCESS DENIED.

- Widely used line of defense against intruders
  - user provides name/**login** (i.e., user ID) and **password**
  - system **compares password** with the **one stored for that specified login**
- The **user ID**: 1) determines that the user is **authorized to access the system**; 2) determines the user's **privileges**; 3) is used in **access control**

The first check is binary: Does this user have an active account? Are they authorized to log in? If the User ID is disabled (for example, an employee who has left the company), access is denied even if the password is correct.

Once you've logged in, the system needs to know what you're allowed to do.

This is directly related to the previous point, but at a granular level (for individual files or resources). When you try to open a file, the system asks: "Does the User ID ' student_123' have permission to edit this specific file?" Your User ID tracks every single action you take within the system to verify your permissions in real time.

Password Authentication

… in the proper way…

(see later)

- Widely used line of defense
  - user provides name/**login** (i.e., user ID) and password
  - system **compares password** with the **one stored for that specified login**
- The **user ID**: 1) determines that the user is **authorized to access the system**; 2) determines the user's **privileges**; 3) is used in **access control**

Password Vulnerabilities

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- offline dictionary attack
- popular password attack
- workstation hijacking
- exploiting multiple password use
- specific account attack
- password guessing against single user
- exploiting user mistakes
- electronic monitoring

Password Vulnerabilities

We're asking ourselves, in general (so not specifically regarding offline dictionary attacks): what is an offline or online attack?

Online vs. Offline attacks

offline dictionary attack

password attack

hijacking

exploiting multiple password use

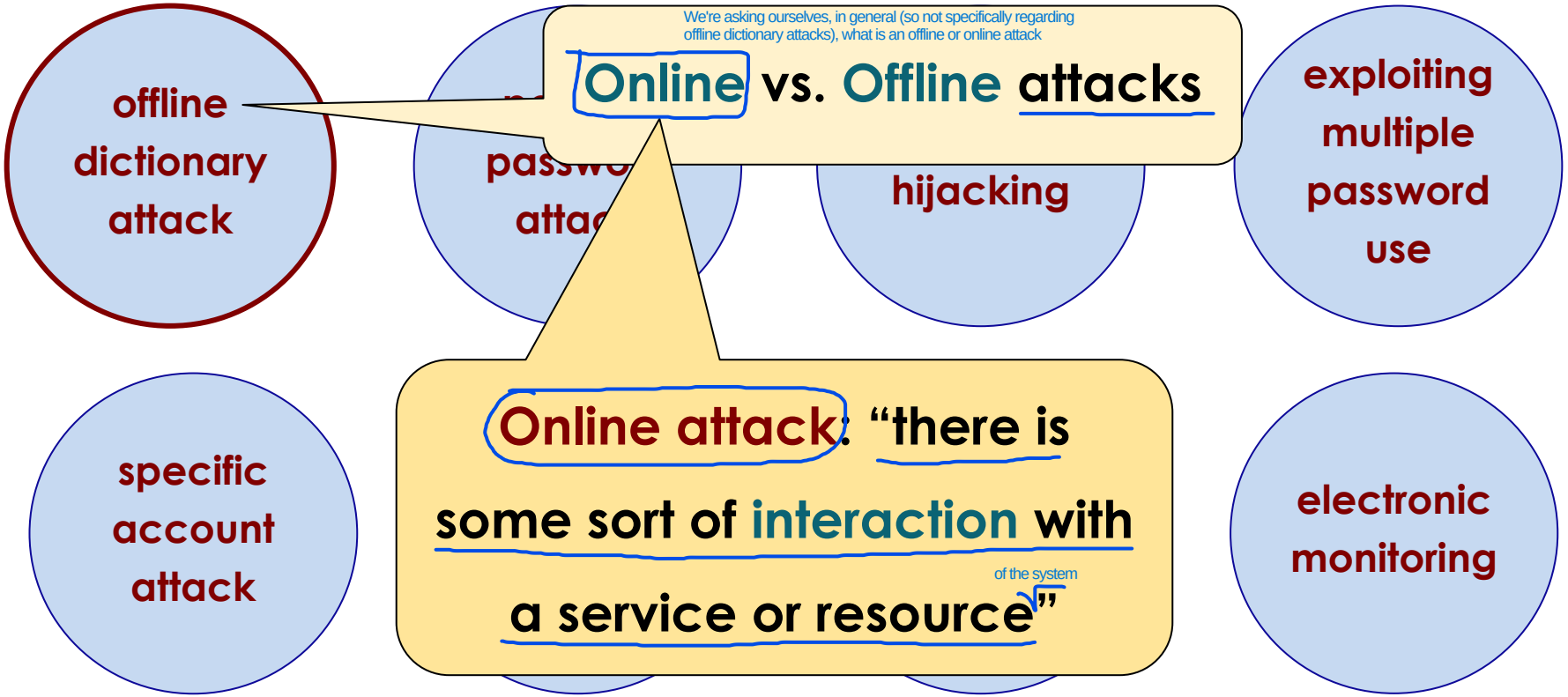
specific account attack

password guessing against single user

exploiting user mistakes

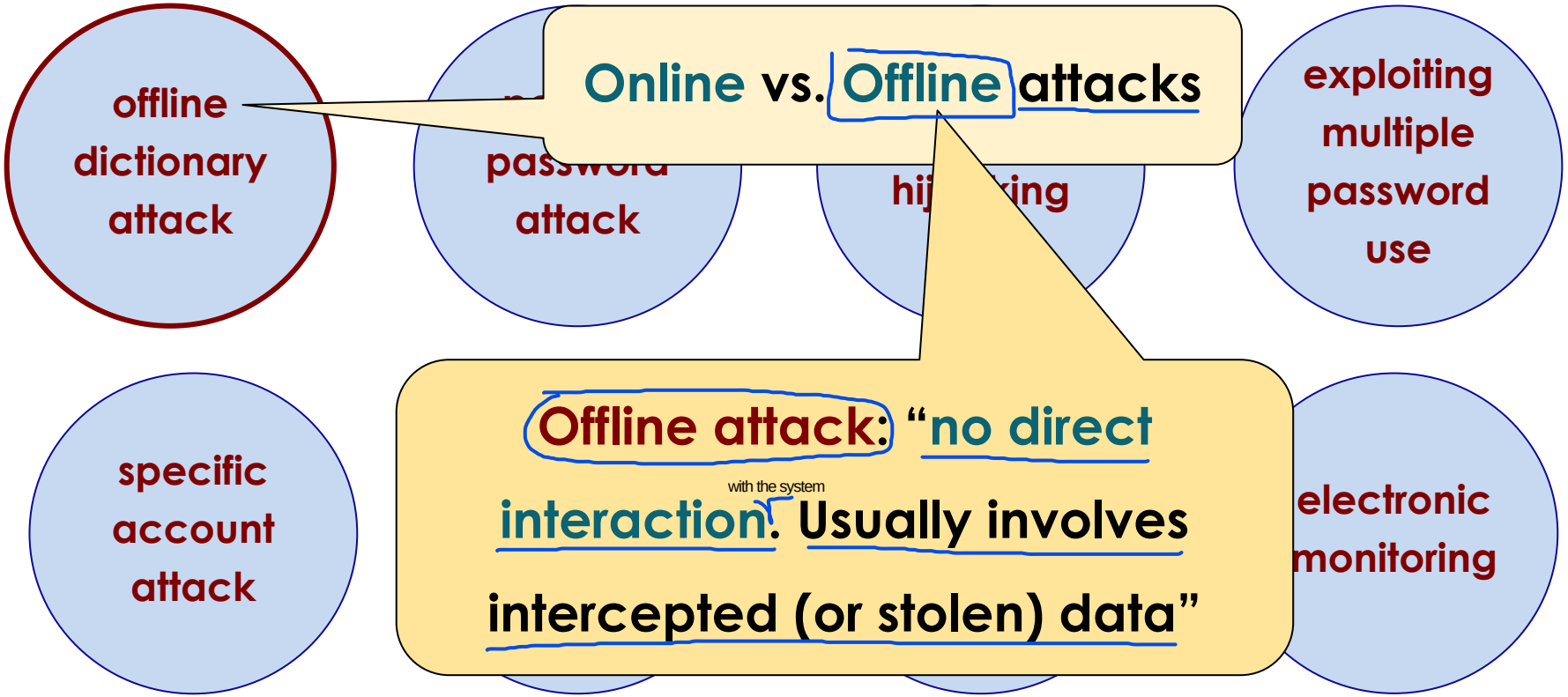
electronic monitoring

Password Vulnerabilities

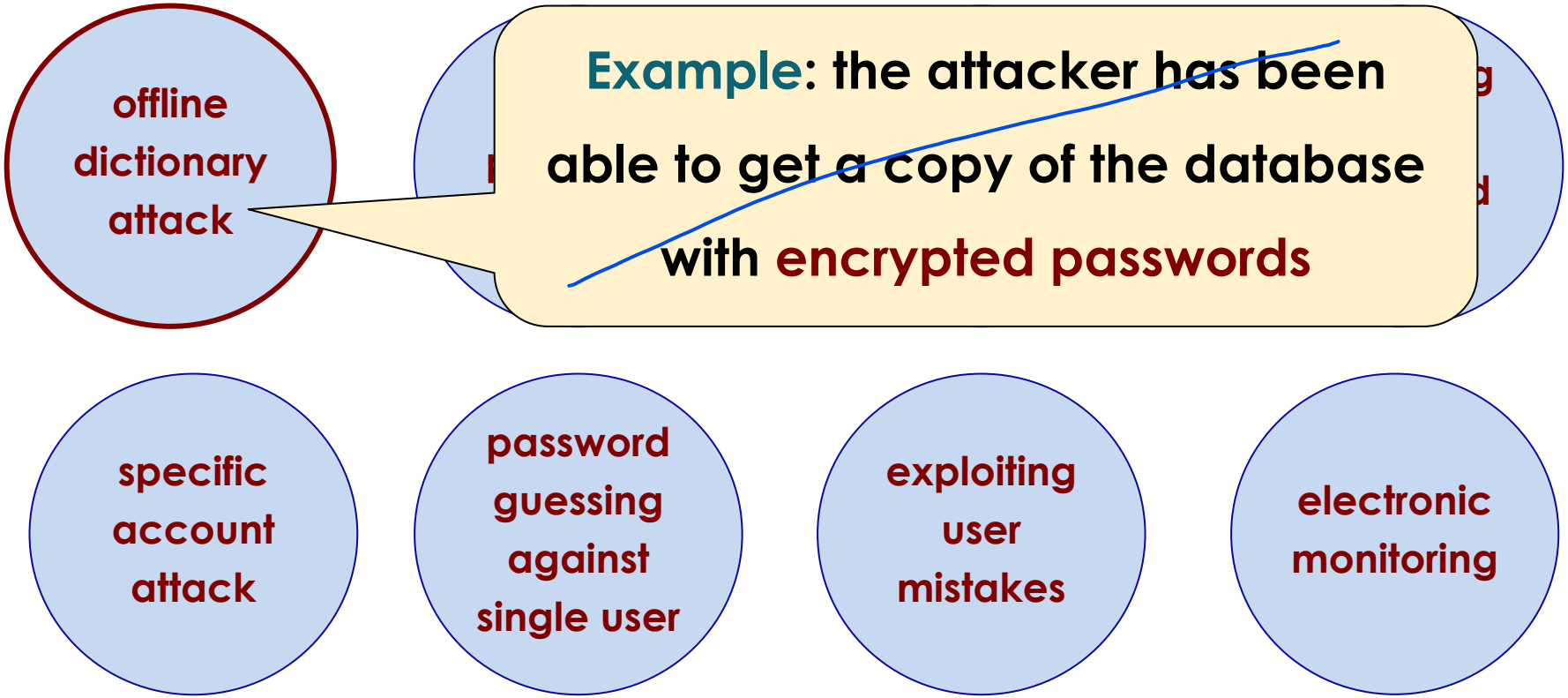


Password Vulnerabilities

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



Password Vulnerabilities



Password Vulnerabilities

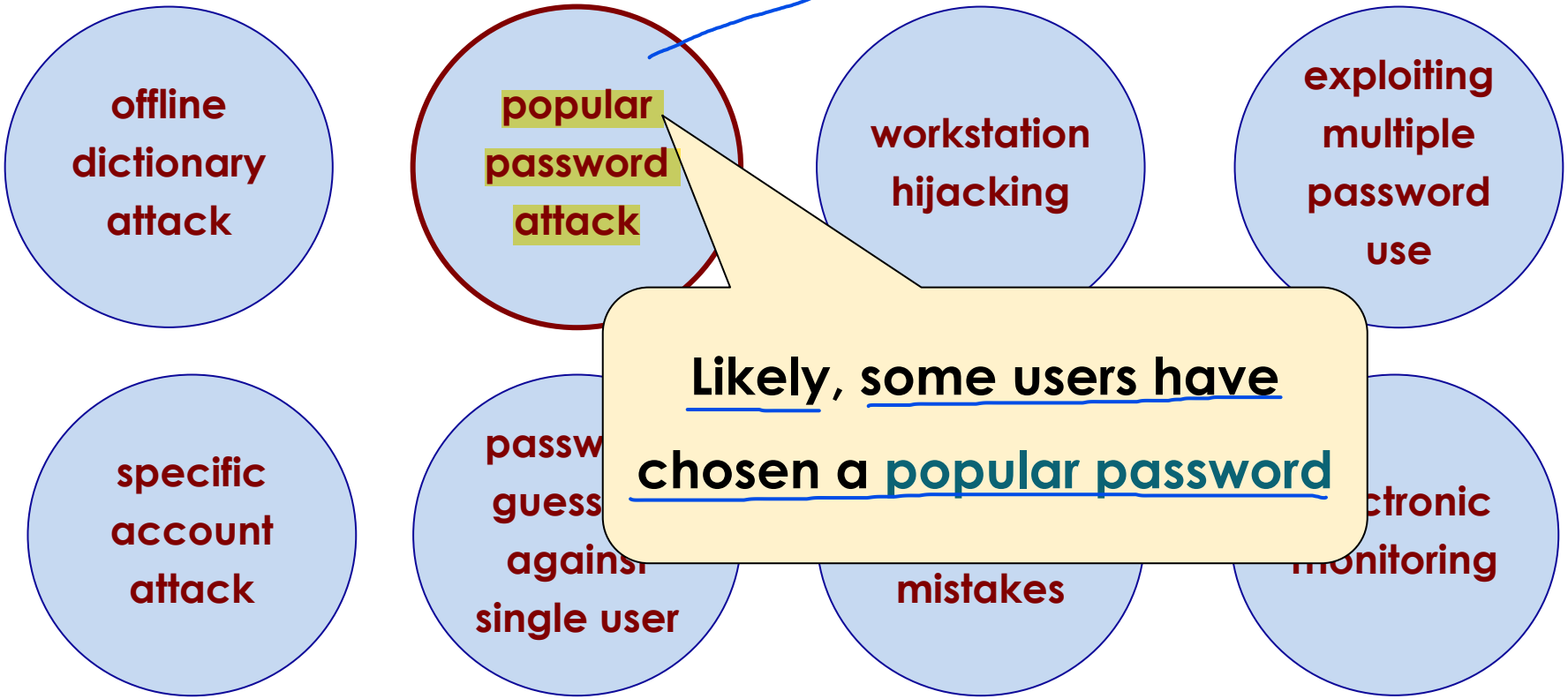
Example: the attacker has been able to get a copy of the database with **encrypted passwords**

He/She will try to find the passwords comparing them with "**common words**" that can be found in a **dictionary**

Password Vulnerabilities

In a popular password attack, the attacker:
- Compiles a list of thousands of different usernames or email addresses (often obtained from past public data breaches).
- Selects a single, extremely common password (the "popular password," such as 'Password123' or "123456").
- Tries that single password on every account in the list, one by one.

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



Password Vulnerabilities

This occurs when authentication is based on passwords, but a user leaves their device unlocked and unsupervised (e.g., during a restroom or launch break). An attacker can then step in and use the active session, completely bypassing the security limitations imposed by password authentication.

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

offline dictionary attack

popular password attack

workstation hijacking

exploiting multiple password use

specific account attack

password guessing against single user

exploiting user mistakes

electronic monitoring

Password Vulnerabilities

Exploiting multiple password use relies on users' bad habit of using the same email and password combination on many different sites. Here's how the attack works:
- The theft: The attacker breaches an insecure site (e.g., a small forum) and steals the database containing user credentials.
- Exploitation: Using automated software, the attacker tries the same stolen credentials on major, well-protected sites (banks, email, social media, corporate networks).

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

offline dictionary attack

popular password attack

workstation hijacking

exploiting multiple password use

specific account attack

password guessing against single user

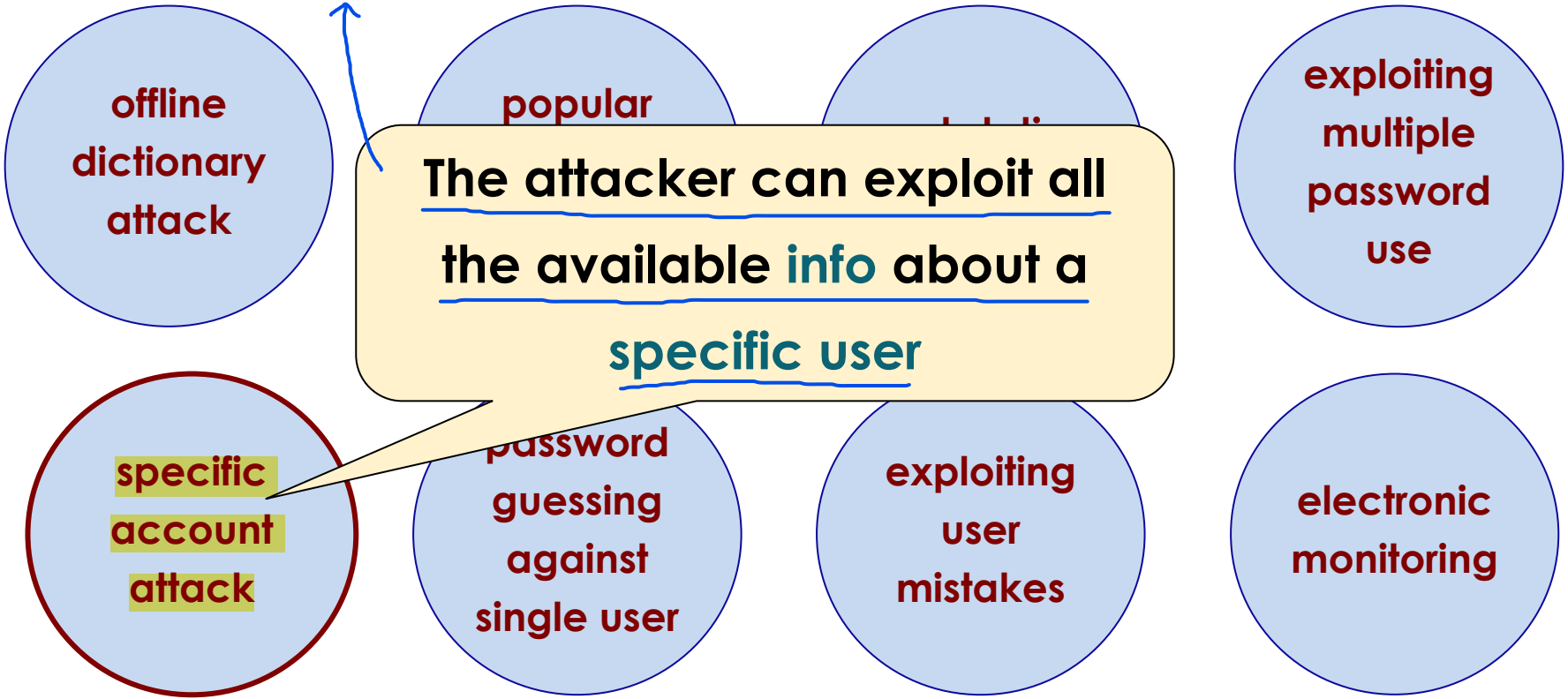
exploiting user mistakes

electronic monitoring

Before attempting to log in, the attacker gathers information about the victim using public sources and social media (a process known as OSINT - Open Source Intelligence). Once the attacker has created a "profile" of the victim, they use this information in two main ways: creating a custom dictionary or bypassing security questions for password resets. Unlike a Popular Password Attack (where a common password is tried on many users), here the effort is focused 100% on a single person. The attack is more effective the more distracted the victim is regarding their privacy on social networks.

Password Vulnerabilities

The key feature of this attack is that the attacker gathers and exploits all available personal information about the victim to guess their password.



Password Vulnerabilities

- Specific Account Attack: This is a larger concept. It includes not only attempting to guess the password, but also the entire process of gathering information about the victim.
- Password Guessing against a Single User: This is the specific action of testing various password combinations against the single username that has been selected as the target.

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

offline dictionary attack

popular password attack

exploiting

In this case the guessing is against a specific user

specific account attack

password guessing against single user

exploiting user mistakes

electronic monitoring

Password Vulnerabilities

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- offline dictionary attack
- popular password attack
- workstation hijacking
- exploiting multiple password use
- specific account attack
- password guessing against single user
- exploiting user mistakes
- electronic monitoring

Password Vulnerabilities

Refers to a category of attacks in which the attacker secretly intercepts the user's actions in real time as the user types or enters their credentials. It is a passive sniffing/eavesdropping that can occur at both the hardware and software levels.

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

offline dictionary attack

popular password attack

workstation hijacking

exploiting multiple password use

specific account attack

password guessing against single user

exploiting user mistakes

electronic monitoring

Countermeasures

1. Strict access
   Controls to prevent **unauthorized access** to **password file**
   → If the attacker cannot download this file, they cannot launch an offline attack.
2. **Intrusion detection** measures
3. **Rapid ~~reissuance~~** reset of **compromised passwords**
   → If the system detects that a user's credentials have been compromised (for example, by finding them in a public data leak on the web), it immediately disables the old password and requires the user to generate a new one on the fly.
4. Account **lockout mechanisms**
   → After 3 or 5 failed attempts, the account is locked, rendering online brute-forcing attacks ineffective.
5. **Policies** to ~~inhibit~~ avoid users from selecting **common passwords**
6. **Training** ~~in~~ and **enforcement** of **password policies**
7. Automatic **workstation logout**
8. **Policies** against **similar passwords** on different **network devices**
   → This way, if a hacker exploits the use of multiple passwords on one device, they will not be able to use similar passwords to compromise the rest of the network.

Countermeasures

- Controls to prevent **ur**
- **Intrusion detection** me
- **Rapid reissuance** of **com** **mised passwords**
- Account **lockout mechanisms**
- **Policies** to inhibit users from selecting **common passwords**
- **Training** in and **enforcement** of **password policies**
- Automatic **workstation logout**
- **Policies** against **similar passwords** on **network devices**

The account is locked after a given number of failed attempts

Countermeasures

- Controls to prevent **ur**
- **Intrusion detection** me
- **Rapid reissuance** of **com** **mised passwords**
- Account **lockout mechanisms**
- **Policies** to inhibit user from se
- **Training** in and **enforcemen**
- Automatic **workstation logou**
- **Policies** against **similar passw**

The account is locked after a given number of failed attempts

It is useful but it is also dangerous... abused (i.e., Denial of Service)

Countermeasures

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

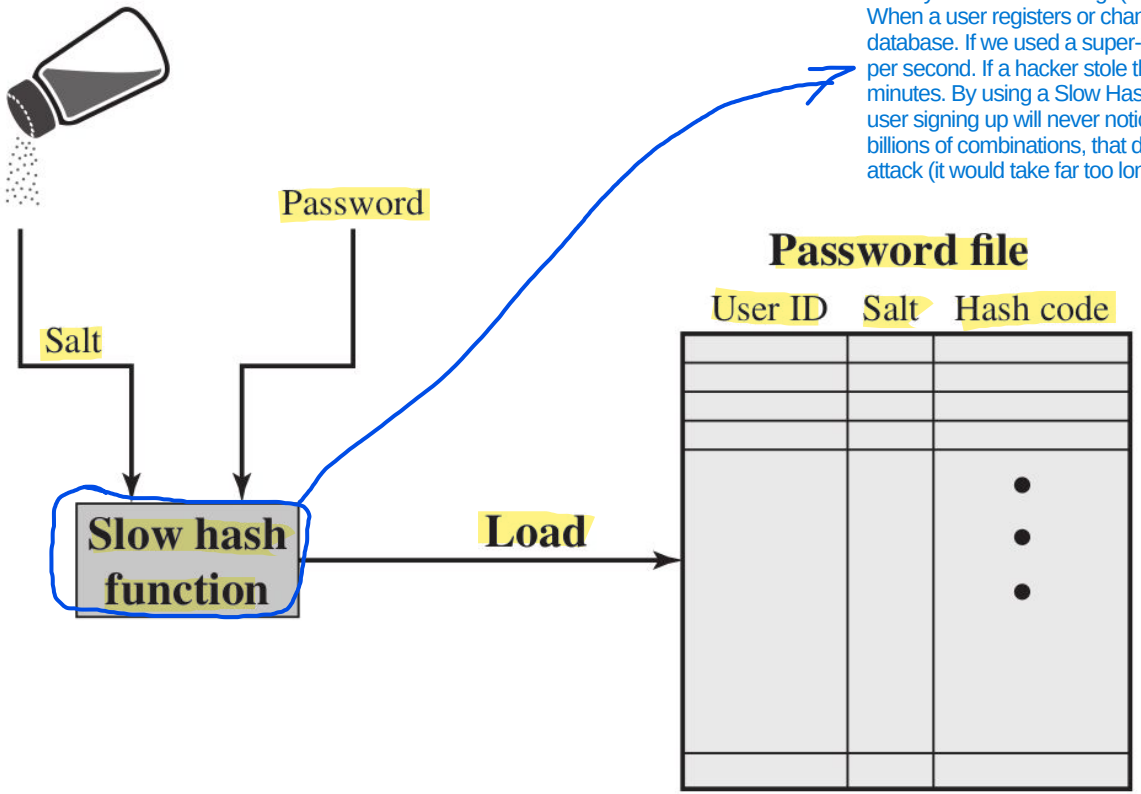
- Controls to prevent **unauthorized access** to **password file**
- **Intrusion detection** measures
- **Rapid reissuance** of **compromised passwords**
- Account **lockout mechanisms**
- **Policies** to inhibit users from selecting **common passwords**
- **Training** in and **enforcement** of **password policies**
- Automatic **workstation logout**
- **Policies** against **similar passwords** on **network devices**

Password Storage

- Storing the passwords "**in clear**" is a **very bad idea** (and… ***passwords delivered by email?***)
- The **sysadmin** would be able to **access all users' passwords** (*and to check if they're used in other systems*)
- An **intruder** would get a **very valuable asset**
- A widely used **password security technique** is the use of **hashed password** and a **salt value**
- This means that the **passwords are never stored "in clear"**

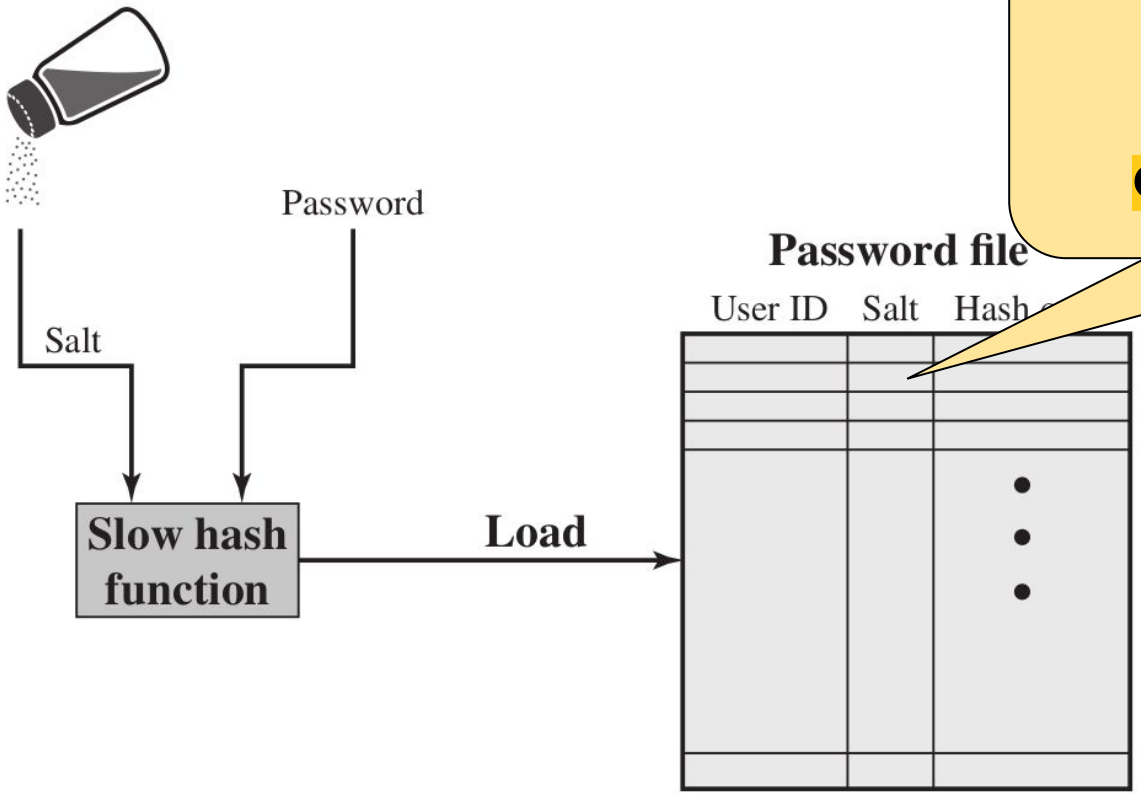
Hashed and Salted Passwords

-> Why does the “Loading” (Registration/Creation) process have to be slow?
When a user registers or changes their password, the system calculates the hash before saving it to the database. If we used a super-fast hash function, a modern computer could calculate billions of hashes per second. If a hacker stole the database, they could crack short or common passwords in just a few minutes. By using a Slow Hash Function, the system intentionally introduces a computational delay. The user signing up will never notice the delay, but for an attacker who has stolen the database and must try billions of combinations, that delay multiplied by each attempt completely destroys the feasibility of the attack (it would take far too long).



Loading a new password

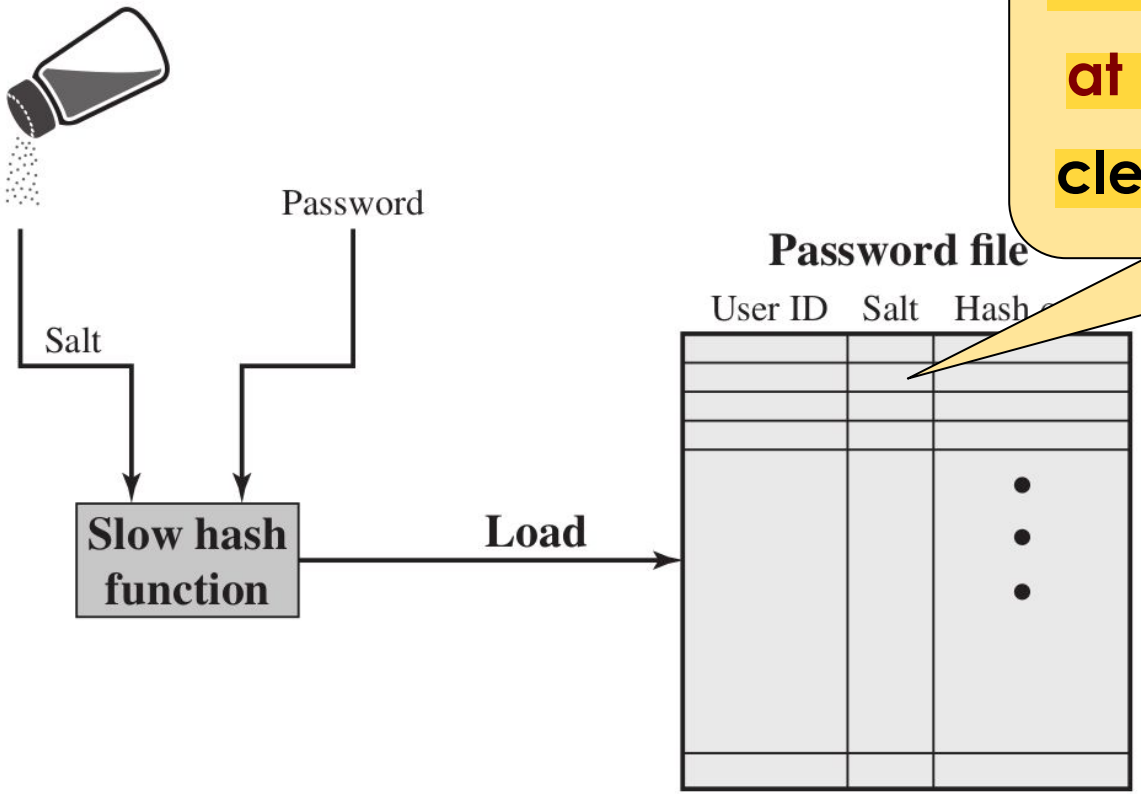
Hashed and Salted Passwords



Each password is obtained through a different salt value

Loading a new password

Hashed and Salted Passwords

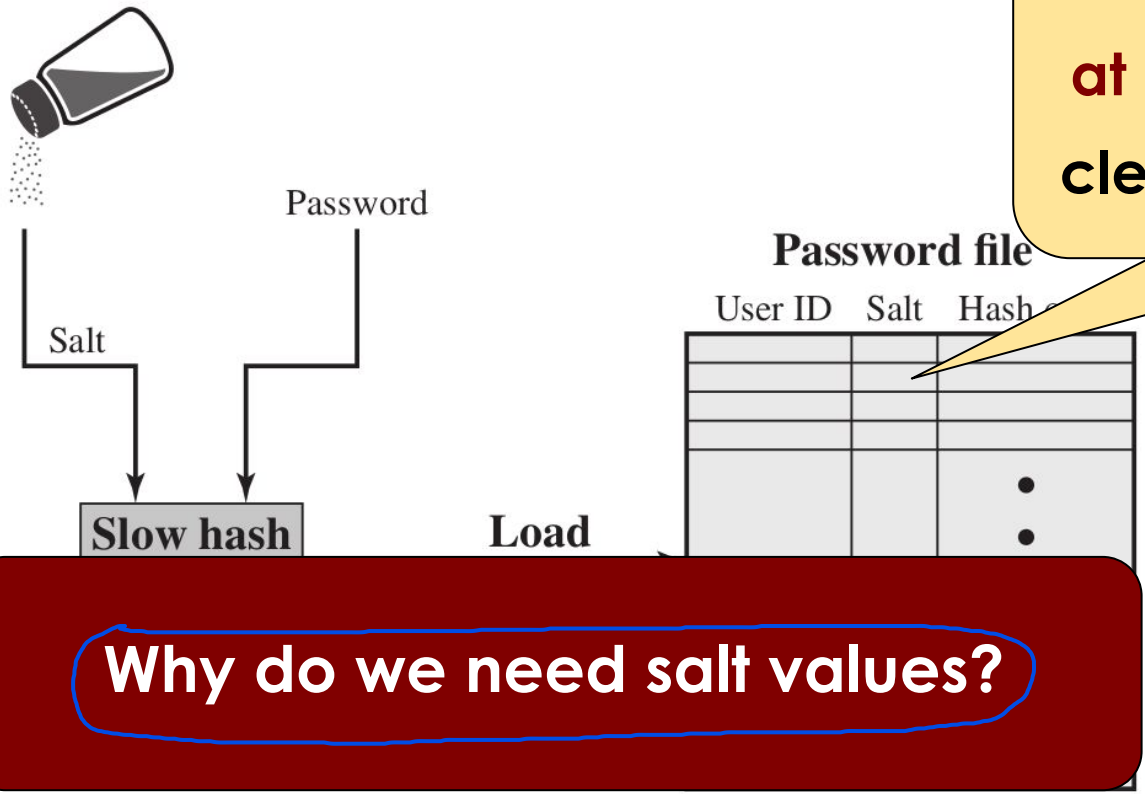


The salt values are chosen at random and stored (in clear) in the password file

Loading a new password

Hashed and Salted Passwords

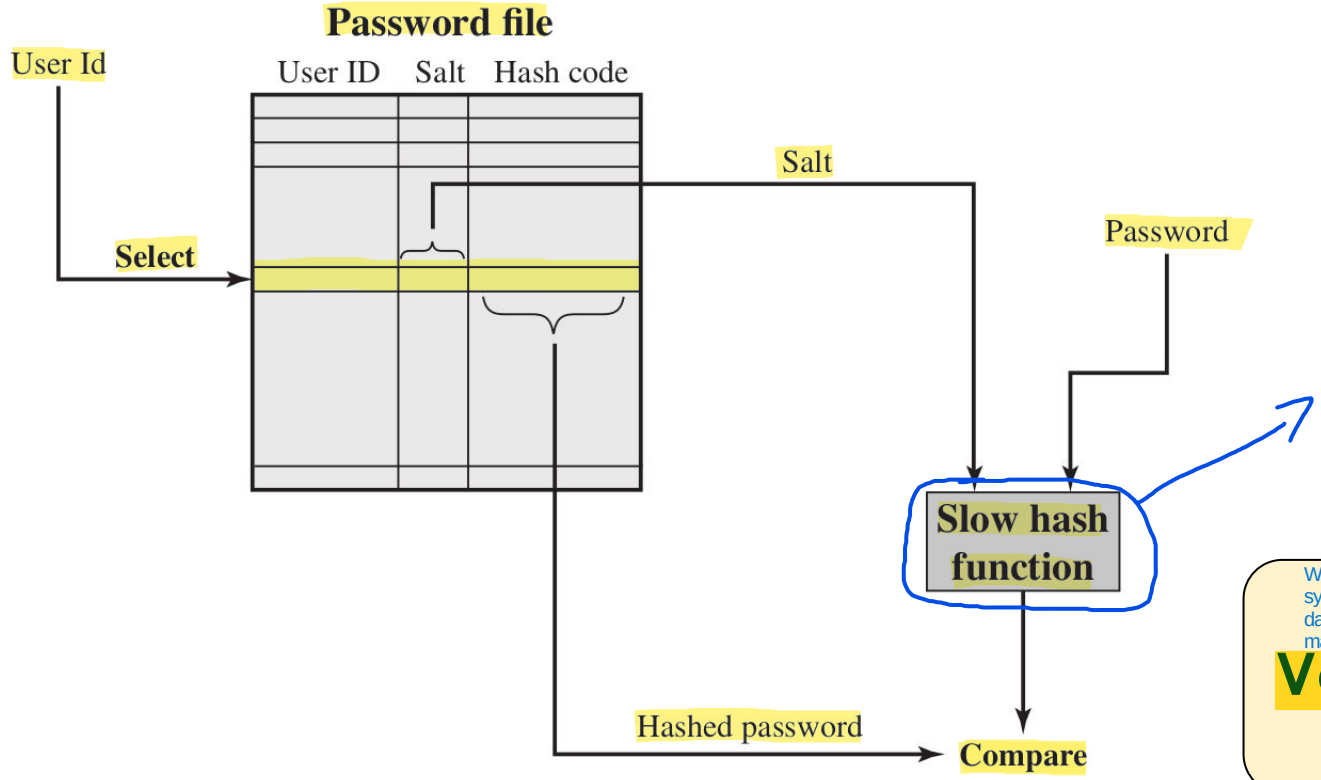
The salt values are chosen at random and stored (in clear) in the password file



Why do we need salt values?

Loading a new password

Hashed and Salted Passwords

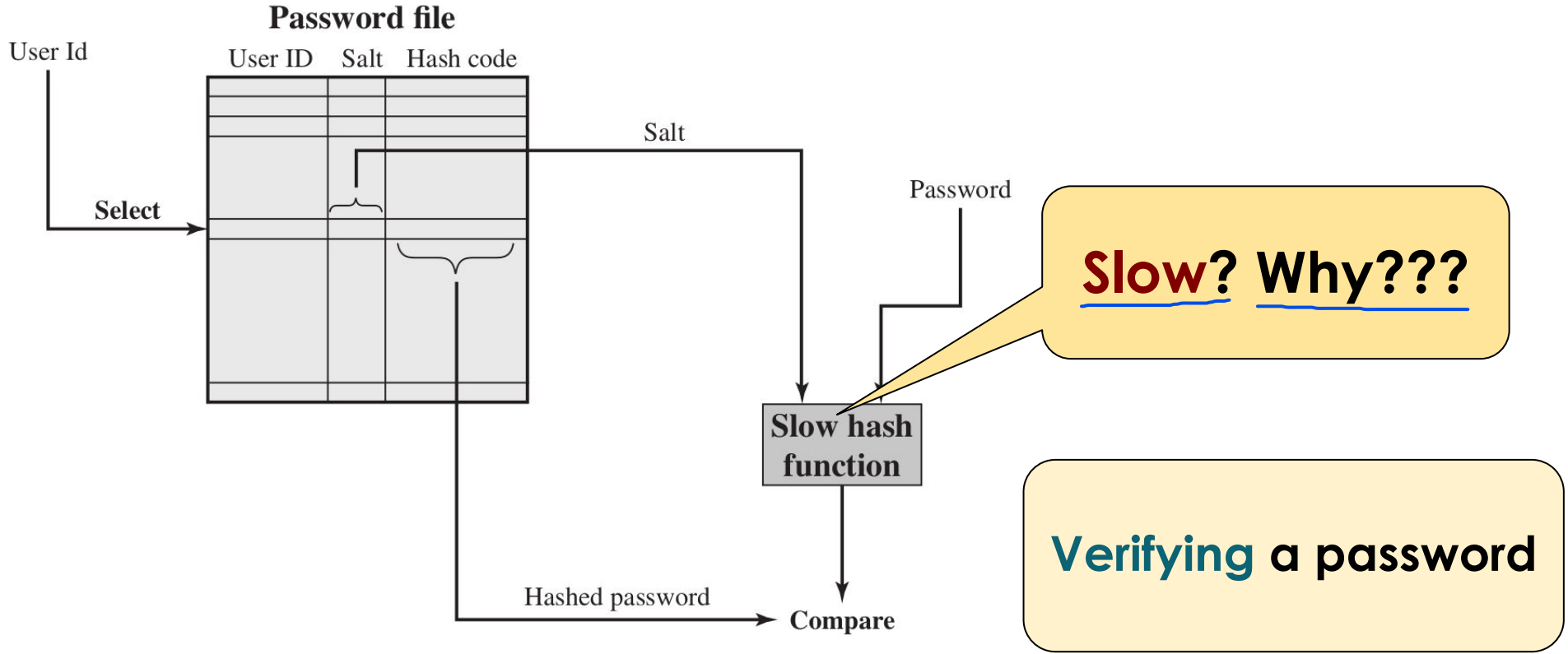


-> Why does the Verify step have to be slow?
Because the algorithm used in the Verifying step must be exactly the same of the Loading step: the server must repeat exactly the same steps to verify if the result matches. This introduces a devastating defensive asymmetry:
- For the legitimate user, waiting 100 ms or 200 ms to log in to the site is perfectly acceptable. The user experience remains excellent.
- For the online attacker, it further slows down their network-based attempts, exhausting their resources.
- For the offline attacker, if the algorithm requires a lot of CPU and memory, the attacker cannot parallelize the attack locally, because the slowness of the function requires hardware resources that GPUs cannot easily scale.

When logging in, the user sends their UserID and password to the system. The system retrieves the user's specific salt value from the database, computes Hash(password + salt), and checks whether it matches the stored hash to verify the user's identity.

Verifying a password

Hashed and Salted Passwords



Hashed Passwords Mechanism

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Thanks to the salt, if two users choose the exact same password, their hashes in the database will be completely different. A system administrator or hacker looking at the file will never see two identical entries and will not be able to tell who has the same password.

- Prevents **duplicate passwords** from being **visible** in the **password file**
- Greatly **increases the difficulty** of **offline dictionary attacks**:
  - for a **salt** of length **b bits**, the **number of possible passwords is increased by a factor of 2^b**
- Becomes nearly impossible to find out whether a person with **passwords on two or more systems** **has used the same password** on all of them

Hashe ism

This is not clear! Why???

- Prevents **duplic passwords** from being **visible** in the **password file**
- Greatly **increases the difficulty** of **offline dictionary attacks**:
  - for a **salt** of length **b bits**, the **number of possible passwords is increased by a factor of 2^b**
- Becomes nearly impossible to find out whether a person with **passwords on two or more systems has used the same password** on all of them

Hashe ism

This is not clear! Why???

- Prevention of Brute-Forcing Attacks: Without a salt, the attacker calculates the hash of a dictionary password only once and compares it simultaneously against all users in the database. With a salt, since each user has a unique, random salt, the final hash is different for everyone, so the attacker is forced to recalculate the entire dictionary from scratch for every single user in the database.
- Rainbow Tables Are Useless: Rainbow Tables are massive databases of pre-computed hashes used by hackers to find passwords in a matter of milliseconds. To still use Rainbow Tables, the attacker would have to generate a table containing every password combined with all 2^b salt variants. This requires storage space that is mathematically and commercially impossible to manage.

- Prevents **duplic passwords** from being **visible** in the **password file**
- Greatly **increases the difficulty** of **offline dictionary attacks**:
  - for a **salt** of length **b bits**, the **number of possible passwords is increased by a factor of 2^b**
- Becomes nearly impossible to find out **passwords on two or more systems ha** of them

To prevent the use of rainbow tables (to be discussed later)

Unix (very old) Implementation

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Original scheme**
  - the password is **up to eight printable characters** in length → If a user entered a longer password, the system ignored everything after the eighth character. This greatly limited the key space.
  - **12-bit salt** used to modify **DES encryption** into a **one-way hash function** → At the time, there were no dedicated hash functions like SHA. Engineers took DES and modified it to function as a one-way hash function. To do this, the 12-bit salt altered the internal workings of DES, ensuring that the standard algorithm itself could not be used to decrypt the data.
  - **zero value** repeatedly **encrypted 25 times** → This was the actual hashing process. The system took an empty string (a sequence of zeros), used the user's password as an "encryption key" (modified by the salt), and encrypted these zeros 25 times in a row. This 25-time loop was intended to intentionally slow down the computation to counter the hardware of the time (it was a primitive Slow Hash Function).
  - output translated to **11 character sequence** → The final result of this process was converted into an 11-character readable string and saved in the password file.
- Now regarded as **inadequate (i.e., really insecure)** Because DES is insecure, salt and hash code is too small
- Sometimes (less and less) still required for **compatibility** with existing account management software or multi-vendor environments

Improved Implementation

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- Recommended hash function was based on **MD5** (now it is **SHA512**)
  - **salt** of up to **48-bits** (or sometimes even more)
  - **password length** is **unlimited**
  - produces **128-bit** hash (MD5), SHA-512 produces 512-bit
  - uses an **inner loop** with **1000 iterations** to achieve **slowdown**

As we saw earlier with Slow Hash Functions, this introduces an intentional delay that drastically slows down hackers' offline brute-forcing attacks.

OpenBSD is a free, open-source, Unix-like operating system

- **OpenBSD** uses Blowfish block cipher based hash algorithm called Bcrypt
  - most (?) secure version of Unix hash/salt scheme
  - uses 128-bit salt to create 192-bit hash value

The main reason is that Bcrypt has an adaptive cost factor. This means that the administrator can decide how many computational rounds the algorithm should perform (e.g., 2^10, 2^12, etc.). As computers become faster over the years, simply increasing this number is enough to keep the algorithm slow and secure, without having to change the software.

Improved Implementation

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- Recommended hash function was based on **MD5** (now it is **SHA512**)
  - **salt** of up to **48-bits** (or sometimes even m e)
  - **password length** is **unlimited**
  - produces **1**
  - uses an **inn** ve **slowdown**
- **OpenBSD** uses algorithm called Bcrypt
  - most (?) secure version of Unix hash/salt scheme
  - uses 128-bit salt to create 192-bit hash value

Broken!!!!

Improved Implementation

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- Recommended hash function was based on **MD5** (now it is **SHA512**)
  - **salt** of up to **48-bits** (or sometimes even more)
  - **password length** is **unlimited**
  - produces **128-bit** hash
  - uses an **inner loop** with **1000 iterations** to achieve **slowdown**
- **OpenBSD** uses Blowfish block cipher based hash algorithm called Bcrypt
  - most (?) secure version of Unix hash/salt scheme
  - uses 128-bit salt to create 192-bit hash value

How do you defend against password cracking? By using a salt, which fights password cracking in two ways: it renders rainbow tables difficult to use and It multiplies the attacker's workload: If two users have the exact same password, they will have two different salts and therefore two completely different hashes in the database. The attacker is forced to perform a separate dictionary attack for each user in the system, drastically slowing down their attack.

Password Cracking

an attacking process used to recover a plaintext password from its hash.

1. **Dictionary attacks**
   - develop a **large dictionary of possible passwords** and **try each of them against the password file**
   - each password must be **hashed using each salt value** and then **compared to stored hash values** (number of hashes: dictionary size * number of users, if we use salt: hashes that must be computationally calculated during attack)
2. **Rainbow table attacks**
   - **pre-compute tables** of **hash values for all salts** → The attacker's ideal goal would be to pre-compute all possible hashes for all passwords by combining them with every existing salt, so that they would only need to perform a quick search in their file at the time of the attack.
   - a **mammoth table** of hash values
   - can be **countered** by using a **sufficiently large salt value** and a **sufficiently large hash length** → The longer the salt, the more the 2^b factor (the total number of salt combinations) explodes mathematically, making the creation of this mammoth table physically impossible due to a lack of storage space and computational time.

Password Cracking

- **Dictionary attacks**
  - develop a **large dictionary of possible passwords** and **try each of them against the password file**
  - each password must be **hashed using each salt value** and then **compared to stored hash values**
- **Rainbow table attacks**
  - **pre-compute tables** of **hash values f**
  - a **mammoth table** of has
  - can be **countered** by us
    large hash length

The salt values are stored (in clear) in the password file

Password Cracking

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Dictionary attacks**
  - develop a **large dictionary of possible passwords** and **try each of them against the password file**
  - each password must be **hashed using each salt value** and then **compared to stored hash values**
- **Rainbow table attacks**
  - **pre-compute tables** of **hash values for all salts**
  - a **mammoth table** of hash values
  - can be **countered** by using a **sufficiently large salt value** and a **sufficiently large hash length**

The introduction of a salt destroys rainbow table attacks for three key reasons:
1) The strength of a rainbow table lies in its universality: an attacker calculates it once for a specific hash function and can use it against any database in the world, because a password will always produce the same hash. With a salt, if two users have the same password, their hashes will be completely different due to their unique salts: the rainbow table becomes useless.
2) To bypass the salt, the hacker would theoretically have to calculate a different rainbow table for every possible salt value. With a 128-bit salt, there are 2^128 possible salt combinations. To store the Rainbow Tables for all these combinations would require such immense storage space that it would take more hard drives than there are atoms in the observable universe: consequently, pre-computing the table becomes mathematically and physically impossible.
3) Without the ability to use pre-computed tables, the attacker loses their advantage and is forced to revert to the dictionary attack, calculating hashes one at a time for every single user in the stolen database. If the database also uses slow hashing functions, the attack becomes so slow that it is inefficient.

Password Cracking

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Dictionary attacks**
  - de s and **try each of them against the**
  - ea **value** and then **compared to stored h es**
- **Rainbow able attacks**
  - **pre-compute tables** of **hash values for all salts**
  - a **mammoth table** of hash values
  - can be **countered** by using a **sufficiently large salt value** and a **sufficiently large hash length**

The goal is to boost the efficiency of the attack

Password Cracking

- **Dictionary attacks**
  - develop a **large dictionary of possible passwords** and **try each of them against the password file**
  - each password r… …nd then **compared to stored hash values**
- **Rainbow table atta…**
  - **pre-comput… …ables** of **hash values for all salts**
  - a **mammoth table** of hash values
  - can be **countered** by using a **sufficiently large salt value** and a **sufficiently large hash length**

It is going to be huge!

Password Cracking

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Dictionary attacks**
  - develop a **large dictionary of possible**
    the password file
  - each password must be **hashed using**
    stored hash values
- **Rainbow table attacks**
  - **pre-compute tables** of **hash values for all sa**
  - a **mammoth table** of hash values
  - can be **countered** by using a **sufficiently large salt value** and a **sufficiently large hash length**

Really too big to be stored

Password Cracking

The Rainbow Table decreases the gap between two inefficient extremes: pure brute-forcing attacks (which require unsustainable computing time) and flat lookup databases (which require impossible amounts of storage space). Its massive efficiency relies on three main principles:
- Shifting the computation phase: the computationally expensive work of calculating billions of mathematical hashes is entirely shifted to the pre-attack phase. Real-time computation during the actual breach is eliminated. Finding a password is reduced to a simple lookup.
- Cryptographic Chain Compression: storing a flat database of every single password-hash pair would require petabytes of storage. Rainbow Tables solve this by grouping passwords into rigid mathematical chains using alternating Hash Functions and Reduction Functions. On the storage, the attacker only stores the first password and the final hash of each chain. If a chain is 1,000 steps long, 99.9% of the data is compressed into thin air, shrinking the total file size while retaining the ability to reconstruct intermediate passwords on the fly.
- Target-Independent Reusability: a Rainbow Table is mathematically bound only to a specific hash function and a defined character set, rather than a specific target system. Once a table is generated, it becomes a permanent, highly reusable asset. The attacker can deploy the exact same table against any system or database in the world that utilizes that specific unsalted hashing function.

- **Dictionary attacks**
  - develop c… **try each of them against the passw…**
  - each pass… …e and then **compared to stored hash val…**
- **Rainbow table …acks**
  - **pre-compute tables** of **hash value…**
  - a **mammoth table** of hash values
  - can be **countered** by using a **sufficiently large salt value** and a **sufficiently large hash length**

The goal is to boost the efficiency of the attack

Can you explain HOW and WHY?

Reduction Chains: Instead of storing every single explicit Password -> Hash pair, the table generates a sequence of alternating hashes and passwords using a Reduction Function (R). Note: The reduction function does not invert the cryptographic hash; it simply maps an output hash value back into a new, valid plain-text string (a new password guess). Space Optimization: For a chain spanning thousands of iterations, the database discards all intermediate values and stores only the first password (Start of Chain) and the final hash (End of Chain) as a single row on the disk. When the attack occurs, simply search for the stolen hash in the "End of Chain" column, and if there is a match, reconstruct that specific chain in a matter of seconds to find the plaintext password.

Password File Access Control

We can block some offline guessing attacks by denying access to encrypted passwords

Countermeasures

1

make available only to privileged users

2

shadow password file: a separate file from the user IDs where the hashed passwords are kept

Even if you implement these countermeasures for the password file, the system is not 100% secure. The slide lists five vulnerabilities that could still allow an attacker to bypass the defences and steal the password file:

vulnerabilities

Privilage escalation by OS vulnerability
weakness in the OS that allows access to the file

accidentally changes the permissions of the shadow file
accident with permissions making it readable

users with same password on other systems

access from backup media

sniff passwords in network traffic

Password Selection Strategies

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- user education
- computer generated passwords
- reactive password checking
- proactive password checking

Password Selection Strategies

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- user education
- computer generated passwords
- reactive password checking
- proactive password checking

users can be told the importance of using hard to guess passwords and can be provided with guidelines for selecting strong passwords

Password Selection Strategies

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- user education
- computer generated passwords
- reactive password checking
- proactive password checking

hard to guess for attackers but users have trouble remembering them

Password Selection Strategies

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- user education
- computer generated passwords
- reactive password checking
- proactive password checking

Computer-generated passwords stored in password managers eliminate the risks associated with the inherent weakness of human-generated passwords (by preventing dictionary attacks and attacks targeting popular passwords). However, this strategy shifts the threat model: the focus of security shifts from protecting individual accounts to protecting the master password and the application itself, which becomes the single point of failure for the entire system.

a (good) possibility is to use password managers: with some attention to the specific threat model

Password Selection Strategies

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- user education
- computer generated passwords
- reactive password checking
- proactive password checking

the system periodically runs its own password cracker to find guessable passwords

Password Selection Strategies

- user education
- computer generated passwords
- reactive password checking
- proactive password checking

user is allowed to select their own password, however the system checks to see if the password is allowable, and if not, rejects it

Password Selection Strategies

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- user education
- computer generated passwords
- reactive password checking
- proactive password checking

In contrast to reactive countermeasures, the proactive approach takes action at the very moment a user is creating or changing their password, preventing them from choosing a weak one.

the goal is to eliminate guessable passwords while allowing the user to select a password that is memorable

Proactive Password Checking

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

3 levels of increasing complexity

1. **Rule enforcement** — The system rejects the password if it does not meet certain quantitative requirements.
   - **specific rules** that passwords **must** ~~**adhere to**~~ follow
2. **Password cracker** — The server acts as an attacker to test the strength of the password in real time.
   - compile a **large dictionary** of passwords **not to use** (+ **variations**)
3. **Other more sophisticated mechanisms** — They represent the state of the art in modern security.
   - **neural networks**-based evaluation — The neural network estimates the actual entropy and predictability of the password chosen by the user, determining whether a brute-forcing attack would take just a few seconds or years to guess it.
   - **password invalidation** based on **public data breaches** — The system connects in real time to global databases that collect all the passwords stolen worldwide in past corporate hacking attacks.

     (e.g., "**Have I Been Pwned**" + **API**)

Password Cracking using AI?

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

ars TECHNICA BIZ & IT TECH SCIENCE POLICY CARS GAMING & CULTURE STORE FORUMS SUBSCRIBE SIGN IN

WANT CHEESE WITH THAT NOTHINGBURGER? —

Meet PassGAN, the supposedly "terrifying" AI password cracker that's mostly hype

AI cracking is on par with conventional methods, but you'd be forgiven for thinking otherwise.

DAN GOODIN - 4/12/2023, 6:22 PM

Aurich Lawson | Getty Images

By processing millions of passwords, AI independently learns the probabilistic distribution of human passwords. It figures out, on its own, which combinations of letters, numbers, and symbols are most likely to be chosen by a real person, mapping the psychology behind password creation.

«These **generative adversarial networks** generate password guesses after **autonomously learning** the distribution of passwords by **processing ~~the spoils of~~** **previous real-world breaches**»

data collected from

Attackers train them using datasets from massive historical data breaches that actually occurred on the web. By studying billions of real passwords chosen by humans, the AI becomes an absolute expert at predicting human patterns.

Password Cracking

Offline password cracking using John the Ripper (JtR)

How does it works?

Check it out!

T6

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Tokens

Until now, you have explored authentication based on “something you know” (passwords). This slide introduces the second category: authentication based on “something you have” (token-based authentication).

Token-based Authentication

Specifically, the table analyzes the technological evolution of the various types of cards used as physical authentication tokens. The evolution is presented from top to bottom, starting with the oldest, purely mechanical technologies and progressing to the most modern and intelligent ones.

Card Type	Defining Feature	Example
Embossed	Raised characters only, on front	Old credit card
Magnetic stripe	Magnetic bar on back, characters on front	Bank card
Memory	Electronic memory inside	Prepaid phone card
Smart Contact Contactless	Electronic memory and processor inside Electrical contacts exposed on surface Radio antenna embedded inside	Biometric ID card

Types of cards used as Tokens

This is the most basic form of card (token): the card contains no electronic components. Authentication and transactions were performed purely mechanically: the card was inserted into a manual device that, by applying pressure to carbon paper, transferred the embossed numbers onto the ticket. From a cybersecurity point of view, this technology is now obsolete.

Token-based Authentication

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Card Type	Defining Feature	Example
Embossed	Raised alphanumeric characters only, on the front side of the card.	Old credit card
Magnetic stripe	Magnetic bar on back, characters on front	Bank card
Memory	Electronic memory inside	Prepaid phone card
Smart Contact Contactless	Electronic memory and processor inside Electrical contacts exposed on surface Radio antenna embedded inside	Biometric ID card

The magnetic stripe on the back contains data stored in the form of magnetic particles. When the card is swiped through the reader, the reader reads the stored static data. The security risk: The data on the magnetic stripe is static and not protected by encryption. Anyone with an inexpensive device (a skimmer) can easily read and duplicate the entire magnetic stripe onto a blank card.

Token-based Authentication

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Card Type	Defining Feature	Example
Embossed	Raised characters only, on front	Old credit card
Magnetic stripe	Magnetic bar on back, characters on front	Bank card
Memory	Electronic memory inside	Prepaid phone card
Smart Contact Contactless	Electronic memory and processor inside Electrical contacts exposed on surface Radio antenna embedded inside	Biometric ID card

and printed (flat or raised)

Unlike magnetic stripes, the data is stored on a memory chip. The reader can read the data and even modify it: these cards contain only hardware memory but lack a processor. The security risk: This means they cannot perform calculations or cryptographic algorithms. Data is simply read or written passively, making them vulnerable to advanced manipulation.

Token-based Authentication

Card Type	Defining Feature	Example
Embossed	Raised characters only, on front	Old credit card
Magnetic stripe	Magnetic bar on back, characters on front	Bank card
Memory	Electronic memory inside	Prepaid phone card
Smart Contact Contactless	Electronic memory and processor inside Electrical contacts exposed on surface Radio antenna embedded inside	Biometric ID card

They contain a real,integrated

Obsolete!

Memory Cards

They have been gradually replaced by smart cards due to several critical vulnerabilities:
1) Lack of processing power.
2) Vulnerability to cloning (skimming): the information they contain is always stored in plain text.
3) Vulnerability to replay attacks.
4) Mechanical and physical deterioration: the magnetic particles degrade over time, are affected by external magnetic fields (demagnetization), and scratch easily. Modern contactless smart cards completely eliminate mechanical friction, last much longer, and offer a better user experience.

- Can **store** but **do not process data**
- The most common type is the **magnetic stripe card**
- Can include an **internal electronic memory**
- Can be used alone for **physical access** (*e.g., hotel rooms*)
- Provides significantly greater security when **combined** with a **password** or **PIN** (*e.g., ATMs*)
- **Drawbacks** of memory cards include:
  - requires a **special reader**
  - **loss** of token — If the user loses the token, they will be locked out of the system until a new one is created.
  - information are often **stored in clear** — Anyone with a compatible reader costing just a few euros (a skimmer) can intercept this data, read it, and copy it onto a blank card, performing a cloning attack.

Memory Cards

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- Can **store** but **do not process data**
- The most common is the **magnetic stripe card**
- Can include an **internal electronic memory**
- Can be used alone for **physical access** (*e.g., hotel rooms*)
- Provides significantly greater security when **combined** with a

Is the locker properly reprogrammed at each room change?

Memory Cards

- Can **store** but **do not process data**
- The most common is the **magnetic stripe card**
- Can include an **internal electronic memory**
- Can be used alone for **physical access** (*e.g., hotel rooms*)
- Provides significantly greater security when **combined** with a

When the card is used on its own (single-factor authentication: something you have). A classic example is a hotel key card. If you find one on the ground, you can use it to enter the room, because the system only checks for the presence of the card, not who is holding it.

Is the locker properly reprogrammed at ~~each~~ every time there is a new guest in the room **~~room change~~?**

NO! So everybody that have access to the memory card can create a copy of it

Sweet dreams :-)

They can store cryptographic keys and execute authentication algorithms directly on the card itself, without ever exposing sensitive data to the outside world.

Token-based Authentication

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Card Type	Defining Feature	Example
Embossed	Raised characters only, on front	Old credit card
Magnetic stripe	Magnetic bar on back, characters on front	Bank card
Memory	Electronic memory inside	Prepaid phone card
Smart	They contain √Electronic memory and processor inside	Biometric ID card
Contact	They have √Electrical contacts exposed on surface → The card must be physically inserted into the reader.	
Contactless	Radio antenna embedded inside	

Smartcards

Two ways in which a smartcard can interact with the outside world or with the user:
- Manual interfaces: some special smart cards feature a tiny keypad and a small liquid crystal display on their physical surface. The user interacts directly with the card (e.g. by entering a PIN) and the card displays a code on the screen.
- Electronic interfaces: the smartcard communicates electronically by sending and receiving digital data from a compatible reader/writer. This electronic interface can be contact-based or contactless (via an integrated NFC/RFID radio antenna).

- **Physical characteristics**: include an **embedded microprocessor**
- **Interface**:
  - **manual interfaces** include a keypad and display for interaction
  - **electronic interfaces** communicate with a compatible reader/writer
- **Authentication** protocol:
  - **static** → The simplest and least secure protocol. The card contains a fixed piece of data (e.g., a certificate or a key) and sends it to the reader. Even though the microprocessor protects the data from immediate cloning, the output sent is always the same.
  - **dynamic** password generator → The smart card functions as an OTP (One-Time Password) generator. Using an internal algorithm based on time or an event counter, the microprocessor generates a one-time password that changes continuously (e.g., every 30 seconds). Even if an attacker intercepts the current password, they won't be able to reuse it because it will expire immediately.
  - **challenge-response** → It is the most secure and advanced protocol. When you insert the smart card, the server sends a challenge to the card, a random number generated on the spot. The smart card's processor takes this number, combines it with the secret key stored inside, and calculates an encrypted response. The response is sent back to the server, which verifies the correctness of the calculation. Why is it exceptional? The secret key never travels over the network and is never revealed to the reader. Furthermore, since the challenge changes with every single login, intercepting the data is completely useless to an attacker (total protection against replay attacks).

Smartcards

- **Physical characteristics**: include an **embedded microprocessor**
- **Interface**
  - **manu** ay for interaction
  - **electronic in** communicate with a compatible reader/writer
- **Authenticati** protocol:
  - **static**
  - **dynamic** password generator
  - **challenge-response**

Not nice for security

Smartcards

- **Physical characteristics**: include an **embedded microprocessor**
- **Interface**:
  - **manual i** …for interaction
  - **electroni** …atible reader/writer
- **Authentication** pr…
  - **static**
  - **dynamic** password generator
  - **challenge-response**

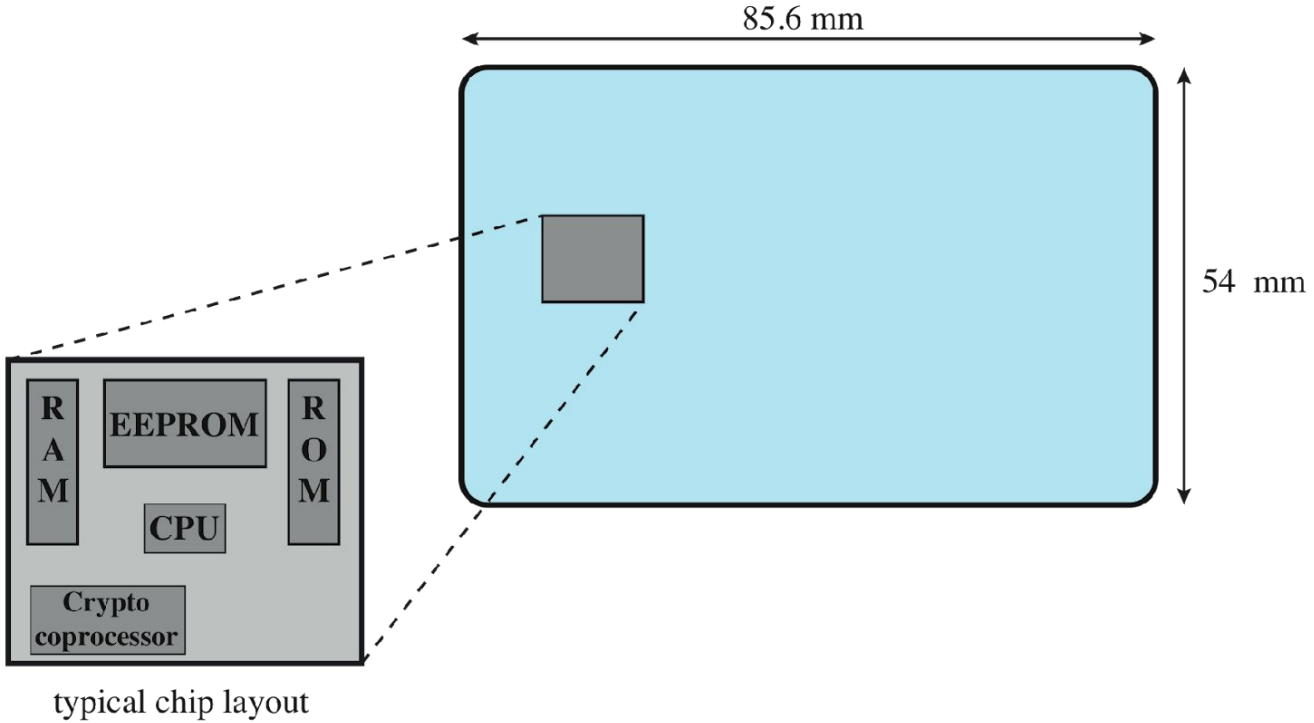
Often, it requires synchronization

Smartcards

- **Physical characteristics**: include an **embedded microprocessor**
- **Interface**:
  - **manual interfaces** include a keypad and display for interaction
  - **electronic interfaces** r/writer
- **Authentication** protoc
  - **static**
  - **dynamic** password ge or
  - **challenge-response**

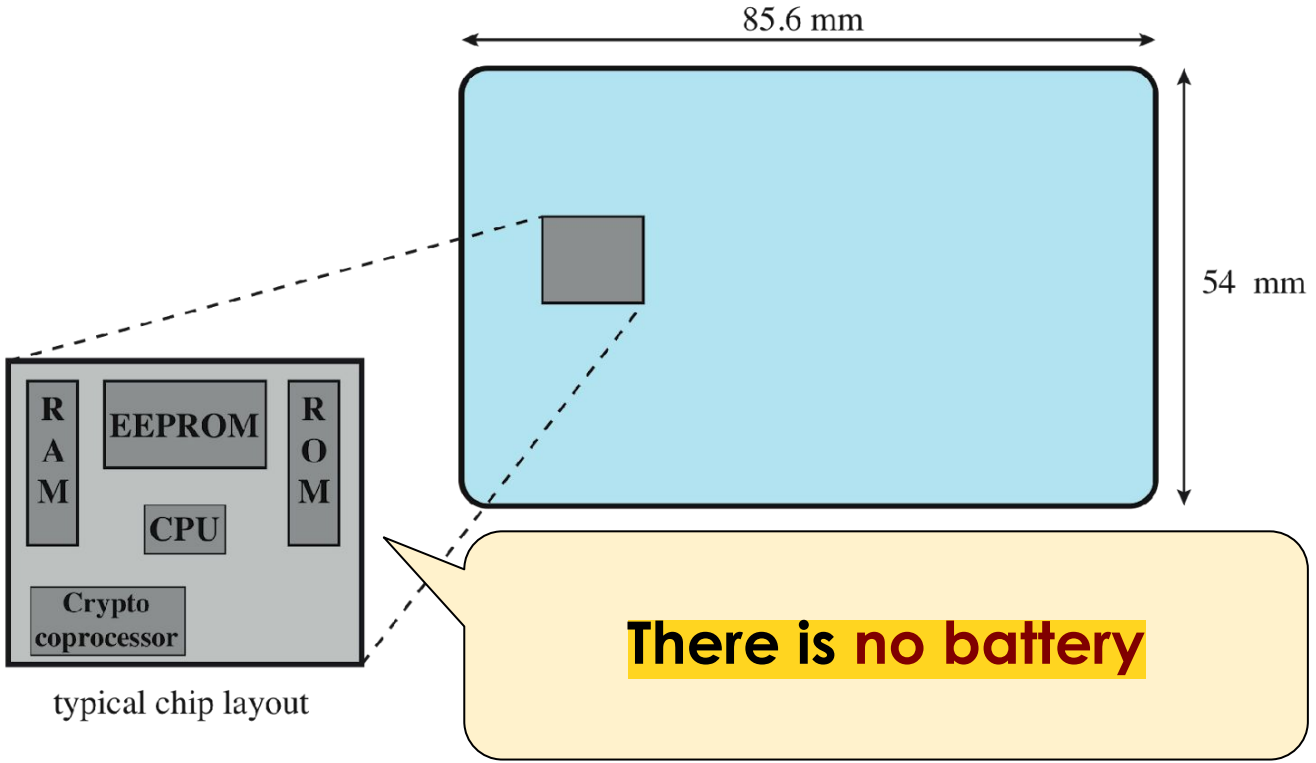
To be discussed later …

Smartcards



typical chip layout

Smartcards


85.6 mm
54 mm
R A M
EEPROM
R O M
CPU
Crypto coprocessor
typical chip layout
There is no battery

Token-based Authentication

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Card Type	De
Embossed	Raised characters onl
Magnetic stripe	Magnetic bar on back
Memory	Electronic memory in
Smart	Electronic memory and processor inside
Contact	Electrical contacts e
Contactless	Radio antenna embedded inside

These cards have no visible contacts because a tiny radio antenna is embedded inside the card. They communicate over very short distances using NFC or RFID technology simply by holding the card near the reader.

Token-based Authentication

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Card	
Embossed	
Magnetic stri	
Memory	lectronic memory ir
Smart Contact Contactless	Electronic memory a Electrical contacts e Radio antenna emb

No major vulnerabilities have been reported

Token-based Authentication

Card	
Embossed	
Magnetic stri	
Memory	lectronic memory ir
Smart Contact Contactless	Electronic memory a Electrical contacts e Radio antenna emb

No major vulnerabilities have been reported

For small value transactions, the PIN is not required

In the world of security, there's a rule of experience: the more secure a system is, the less usable it is; the more usable it is, the less secure it is. Contactless payment is the perfect example: to pay for your coffee at the café, all you have to do is hold your card or phone up to the reader for a fraction of a second. You don't have to insert the card into the reader, you don't have to enter a PIN, and you don't have to sign anything. It's fast and frictionless. The Security Risk: If you lose your card or a thief steals it, they too can go to the café and pay for a coffee simply by holding it up, because the underlying system doesn't verify who is holding the card. Furthermore, theoretically, an attacker with a portable POS reader hidden in a backpack could approach you in a crowded subway and skim money from your jacket. To maintain high usability without compromising security, designers had to find a compromise (a trade-off): for example, allowing contactless payments without a PIN only below a certain threshold (e.g., up to €50).

Token-based Authentication

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- There is a trade-off between usability and security
- Security is partially based on "usage pattern analysis"

When the Usage Pattern Analysis algorithms detect an anomaly compared to your historical profile, the system reacts immediately as a security measure: it either rejects the contactless transaction, forcing the cardholder to insert the card and enter the PIN, or temporarily blocks the card by sending an SMS or app notification to the actual owner.

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

NON RIENTRA NEL PROGRAMMA D'ESAME

Remote Authentication

Remote (User) Authentication

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Authentication** over a **network**, the Internet, or a communications link **is more complex** (vs. local)
- **Additional security threats** such as:
  - **eavesdropping**, **capturing** a password, **replaying** an authentication sequence that has been observed (and recorded by an attacker)
- Generally relies on some form of a **challenge-response protocol** to counter threats

Remote (User) Authentication

Client

Host

U, User

U

r, random number
h(), *f*(), functions

(*r*, *h*(), *f*())

P′
r′, return of *r*

f(*r′*, *h*(*P′*))

if *f*(*r′*, *h*(*P′*)) =
f(*r*, *h*(*P*(*U*)))
then yes else no

yes/no

Remote (User) Authentication

Client

Host

password

nonce

U, User

U

r, random number
h(), *f*(), functions

(*r*, *h*(), *f*())

P′
r′, return of *r*

f(*r′*, *h*(*P′*))

r' = r

if *f*(*r′*, *h*(*P′*)) =
f(*r*, *h*(*P*(*U*)))
then yes else no

yes/no

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

NON RIENTRA NEL PROGRAMMA D'ESAME

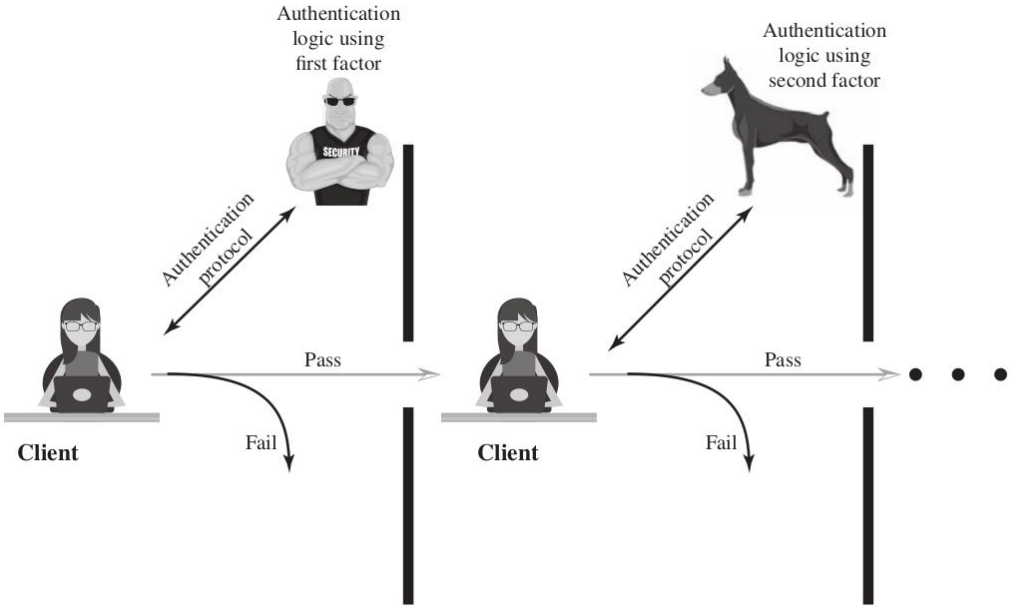
Multi-Factor Authentication

Multi-factor Authentication

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- It refers to the use of **more than one** of the **authentication means**

- Implementations that use **multi-factors** are considered to be **stronger** than those that use only one factor

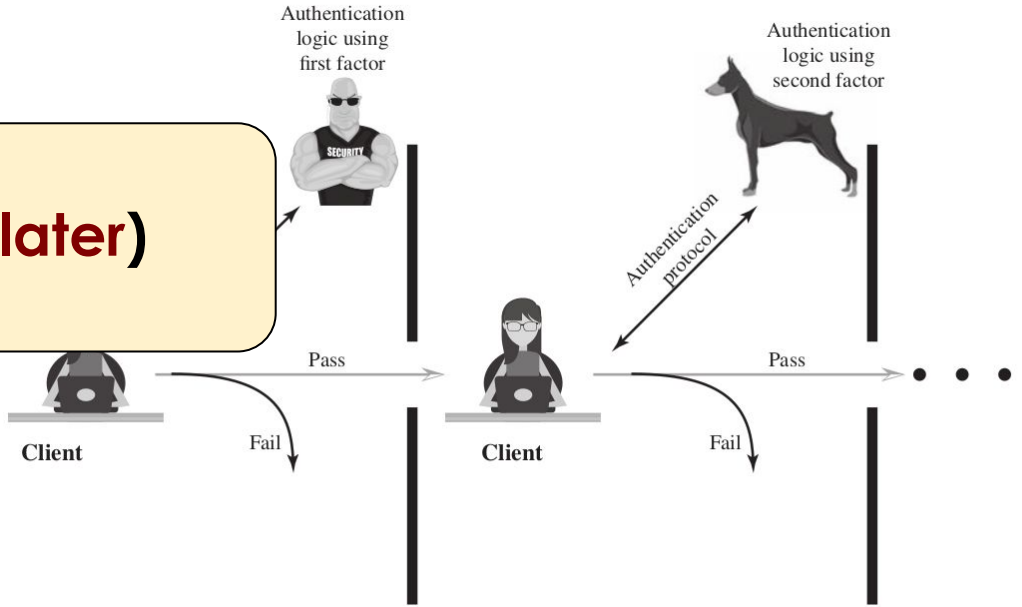


Multi-factor Authentication

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- It refers to the use of **more than one** of the **authentication means**
- Implemen… **multi-factor** … considered to be **stronger** than those that use only one factor

Yes but … (see later)



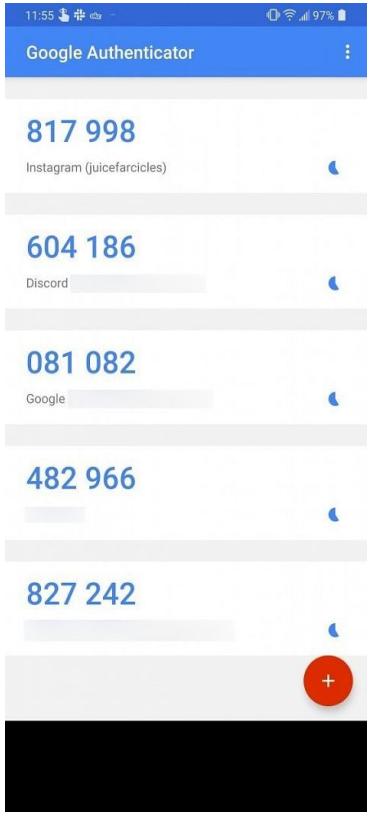
Example: two-step authentication

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Two** (or more) **steps** are used to **verify the identity** of an entity that is trying to access services
- **Possible implementation (both steps are mandatory)**:
  a. something that the **users knows** (e.g., username + password)
  b. something that is **generated by a device owned by the user** (e.g., a dynamic PIN), or a **one-time code** delivered by **SMS**

Example: two-step authentication

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



Google Authenticator

Ex…thentication

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Tw…** **the identity** of an entity that is try…
- **Po…** **…re mandatory)**:
  a. …sername + password)
  b. **…devic…ned by the user** (e.g., a dynamic PIN), or a **one-time code** delivered by **SMS**

“Out of band verification using SMS is deprecated, and will no longer be allowed in future releases of this guidance” (NIST)

Ex…thentication

- **Tw…** …**the identity** of an entity that is try…
- **Po…** …**re manc…**
  a. …sernam… …sword)
  b. …**devic…** …**ned b…** …**he user** (e.g., a dynamic PIN), or a **one-time code** delivered by **SMS**

"Out of band verification using SMS is deprecated, and will no longer be allowed in future releases of this guidance" (NIST)

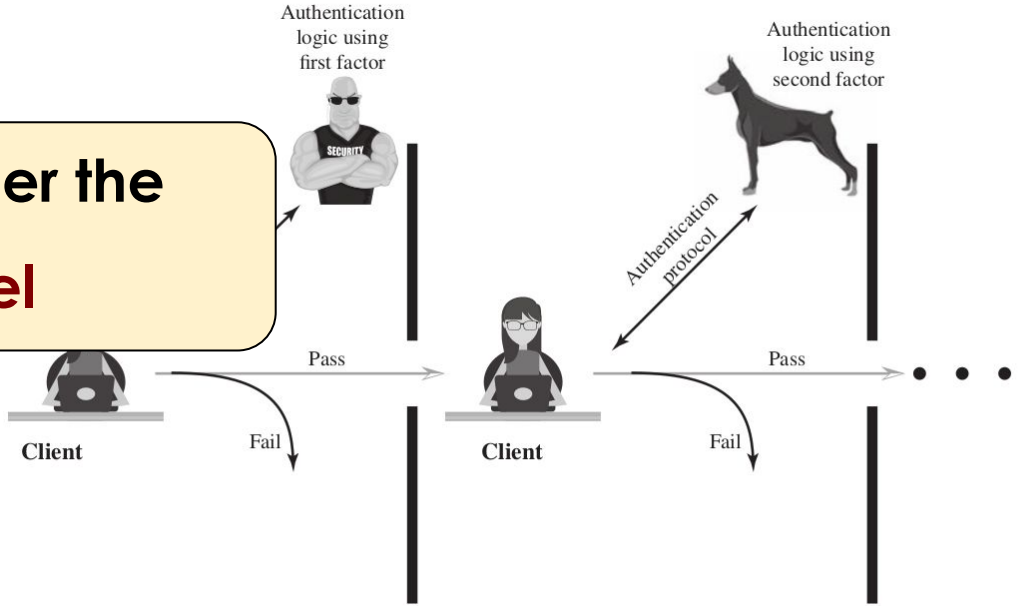
SMS are not good for security

Multi-factor Authentication

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- It refers to the use of **more than one** of the **authentication means**
- Implemen… **multi-factor** … considered to be **stronger** than those that use only one factor

Carefully consider the threat model

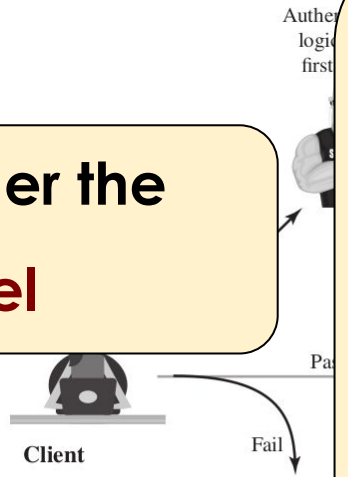


Multi-factor Authentication

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- It refers to the use of **more than one** of the **authentication means**
- Implemen… **multi-factor**… considered to be **stronger** than those that use only one factor

Carefully consider the threat model



For example: the security of the channel used for the 2nd step verification

Multi-factor Authentication

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- It refers to the use of **more than one** of the **authentication means**
- Implemen… **multi-factor**… cons… be **stronger** than thos… use only one factor

Carefully consider the threat model

Is it IN band or OUT of band?

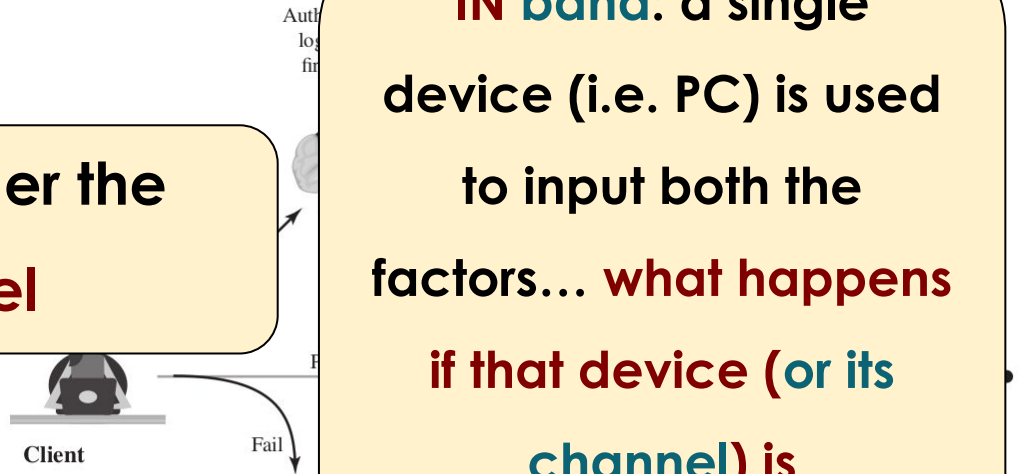
For example: the security of the channel used for the 2nd step verification

Multi-factor Authentication

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- It refers to the use of **more than one** of the **authentication means**
- Implemen… **multi-factor**… considered to be **stronger** than those that use only one factor

Carefully consider the threat model



IN band: a single device (i.e. PC) is used to input both the factors… what happens if that device (or its channel) is compromised?

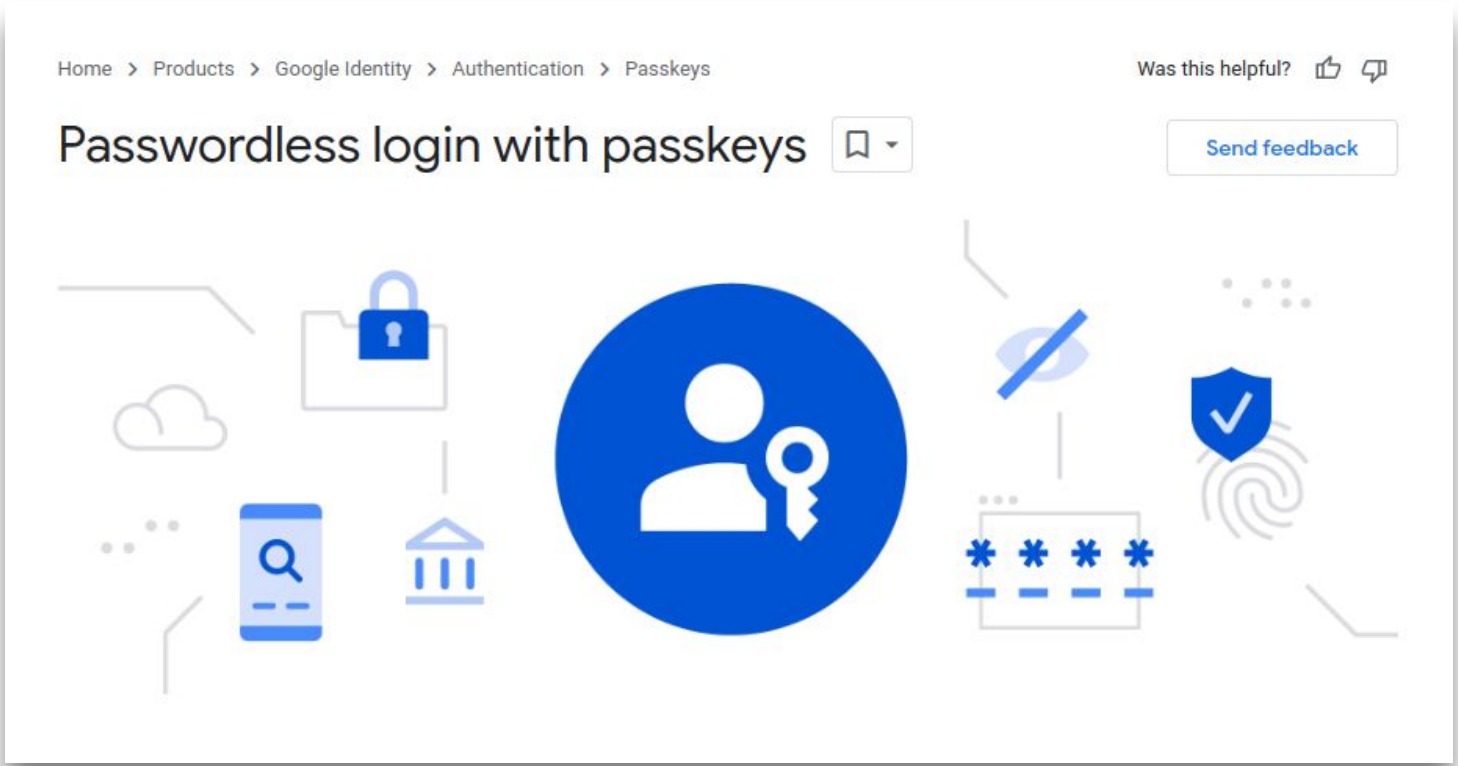
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

NON RIENTRA NEL PROGRAMMA D'ESAME

Passkeys

The next step… passkeys

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



Passkeys: technical description

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- The goal is to **completely replace** traditional **username** and **passwords**
- Passkeys are based on **public-key cryptography** (i.e., key pairs)
- Passkeys are stored on the **local storage of personal devices**, **not on remote servers**. This can reduce the impact of **server-side data breaches**
- **Phishing resistant**: each passkey is solely linked to the specific app or website it was created for

Passkeys: technical description

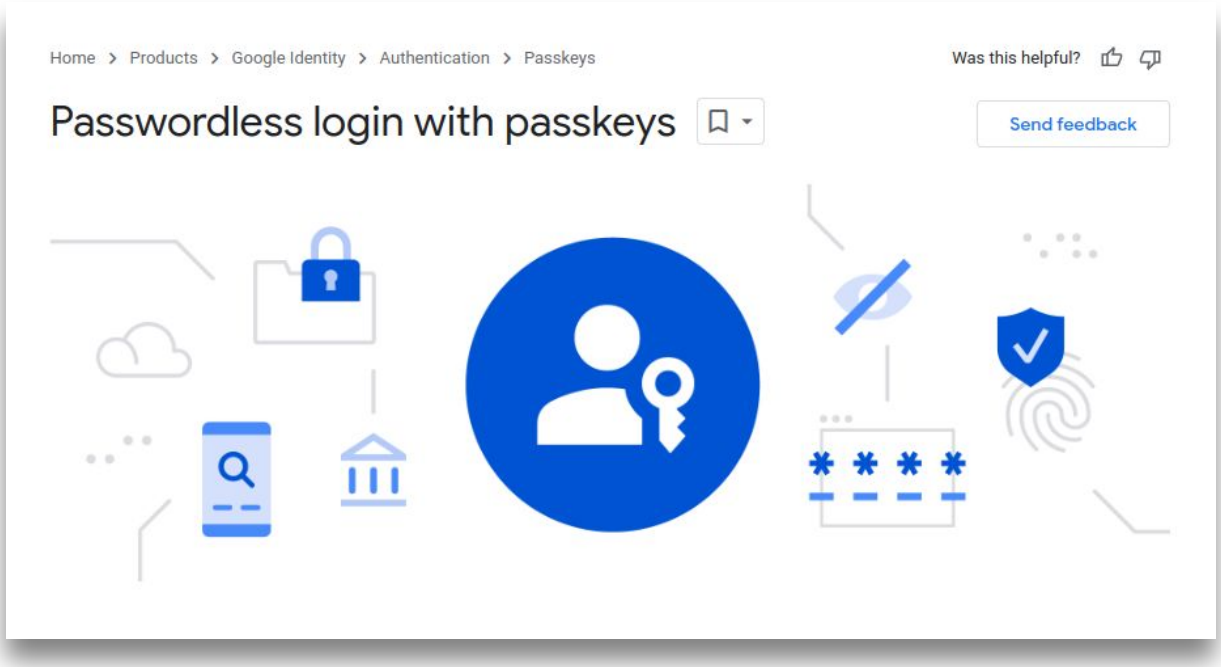
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- Passkeys utilize **biometric verification** (i.e., fingerprint or facial recognition) for authorizing logins
- **Much better usability** than password
- Passkeys can be **synced between different devices** using the appropriate tools and considering the **effects on security** of this operation

The next step… passkeys

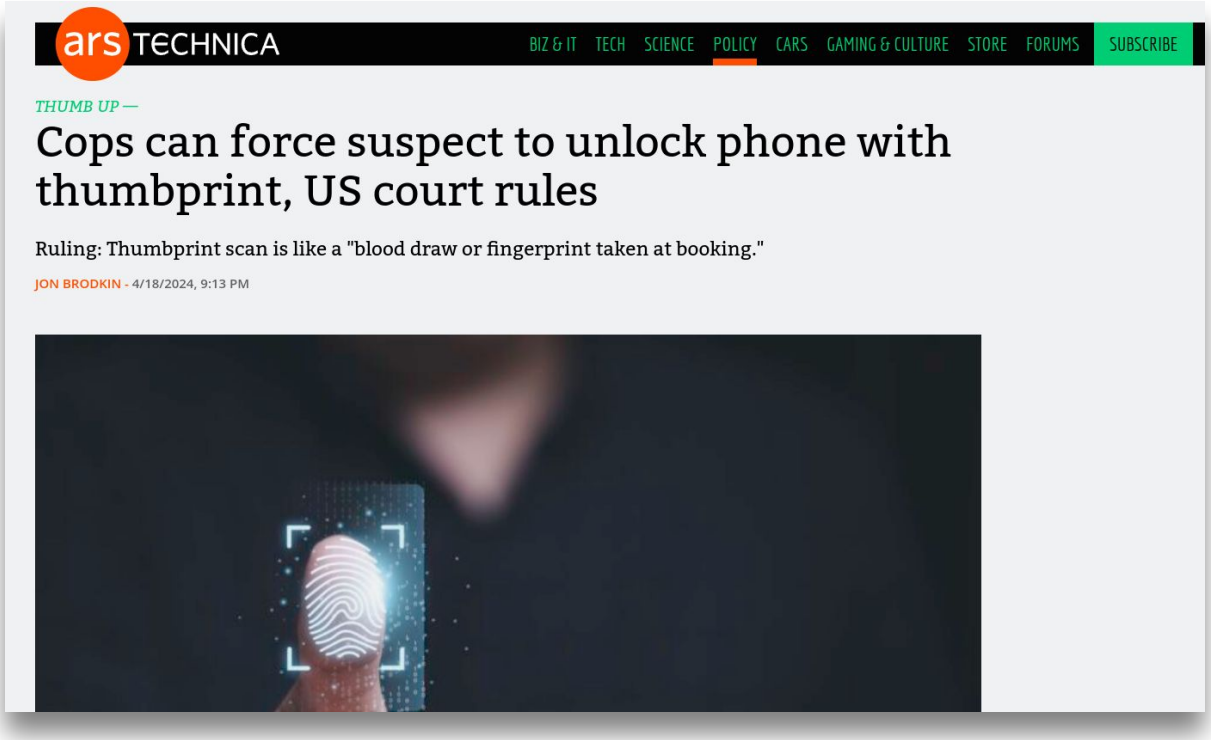
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- **Only partially supported** and **deployed**
- Even if **standard**, there are relevant limitations in terms of **interoperability**
- What about the **threat model**?



Legal implications of biometric auth

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



- Authentication-based on **biometry** can have **different legal protection** than **passwords**
- What about the protection of **passkeys**?

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

CyberSecurity

Gabriele D'Angelo

<g.dangelo@unibo.it>

http://www.cs.unibo.it/gdangelo/

Attributions and Notes

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- Most of the slides are from: "**Computer Security: Principles and Practice**", 4th edition, W. Stallings and Lawrie Brown
- Some others are from: Dr. **Mehmet H. Gunes** (Department of Computer Science & Engineering - University of Nevada, Reno)
- Some others are from: **Mario Cagalj**, Ph.D. (Faculty of Electrical Engineering, Mechanical Engineering and Naval Architecture (FESB), University of Split)
- I wish to **thank all of them**, **any errors that remain are my sole responsibility**